

03-03-2026

Lab - 14

Lab 4: Convolutional Neural Networks (CNNs) and Image Loading -

Objective: In previous labs, we loaded datasets directly into memory using PyTorch's built-in datasets. In the real world, you will usually receive a hard drive or folder full of images. In this lab, you will:

1. Load image files directly from a disk directory structure.
2. Build a Convolutional Neural Network (CNN) to process 3-channel (RGB) color images.
3. Compare the spatial feature extraction of CNNs to the MLPs used in previous labs.

```
In [ ]: import torch
import os
import torchvision
from PIL import Image
from tqdm import tqdm

def setup_disk_dataset(root_dir='./data/cifar10_disk'):
    if os.path.exists(root_dir):
        print("Dataset already exists on disk!")
        return

    print("Saving CIFAR-10 images to disk folder structure...")
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True)

    for i, (img, label) in enumerate(tqdm(trainset)):
        class_name = trainset.classes[label]
        save_dir = os.path.join(root_dir, 'train', class_name)
        os.makedirs(save_dir, exist_ok=True)
        img.save(os.path.join(save_dir, f'img_{i}.png'))
    print(f"Images saved successfully to {root_dir}/train!")

setup_disk_dataset()
```

Task 1: Define Data Transformations

When loading images from disk, they are read as PIL Images. We need to convert them to PyTorch Tensors and normalize them.

Task: Use `torchvision.transforms.Compose` to convert the images to tensors and normalize them with a mean and standard deviation of 0.5 across all three RGB channels.

```
In [ ]: # Gradient calculation easy pade ena mate
import torchvision.transforms as transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

Task 2: Load Dataset from Disk

PyTorch provides `torchvision.datasets.ImageFolder` to easily load images that are organized in folders named after their classes.

Task: Use `ImageFolder` to load the training dataset from the `./data/cifar10_disk/train` directory, applying the transform you defined in Task 1.

```
In [ ]: from torchvision.datasets import ImageFolder

train_dataset = ImageFolder(
    root='./data/cifar10_disk/train', transform=transform)
```

```
In [ ]: len(train_dataset)
```

```
In [ ]: train_dataset.classes
```

Task 3: Create the DataLoader

Task: Wrap your `train_dataset` in a `torch.utils.data.DataLoader`. Set a batch size of 64 and ensure the data is shuffled.

```
In [ ]: from torch.utils.data import DataLoader

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

print(len(train_loader))
```

Task 4: Build the CNN Architecture

Unlike MLPs that flatten the image immediately, CNNs use convolutional layers to extract spatial features.

Task: Create a class `SimpleCNN` that inherits from `nn.Module`. Your network should have:

1. A convolutional layer (`nn.Conv2d`) taking 3 input channels, outputting 16 channels, with a kernel size of 3.
2. A ReLU activation.
3. A Max Pooling layer (`nn.MaxPool2d`) with a kernel size of 2 and stride of 2.

4. A fully connected layer (`nn.Linear`) that maps the flattened pooled features to 10 output classes. (Hint: CIFAR-10 images are 32x32. Calculate the spatial dimensions after convolution and pooling to find the input size for the Linear layer).

```
In [ ]: import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3)
        # Out -> 16 * 30 * 30
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        # Out -> 16 * 15 * 15
        self.fc1 = nn.Linear(16 * 15 * 15, 10)
        # Out -> 10

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = x.view(-1, 16*15*15)
        x = self.fc1(x)
        return x

model = SimpleCNN()
print(model)
```

Task 5: Define Loss and Optimizer

Task: Initialize the Cross-Entropy Loss function and the Adam optimizer with a learning rate of 0.001.

```
In [ ]: import torch.optim as optim

optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
```

Task 6: The Training Loop

Task: Write a training loop for 5 epochs. Iterate over the `train_loader`, perform the forward pass, calculate the loss, backpropagate, and update the weights. Print the loss at the end of each epoch.

```
In [ ]: def train():
    model.train()

    num_epochs = 5
    running_loss = 0

    for epoch in range(num_epochs):
        print(f"\n--- Epoch {epoch + 1}/{num_epochs} ---")
```

```

    for batch_No, (data, target) in enumerate(train_loader):

        output = model(data)

        loss = criterion(output, target)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(
        f"Epoch {epoch + 1} completed! Loss : {running_loss / len(train_dataset)}

```

```
In [ ]: train()
```

Test

```

In [ ]: import torchvision

test_dataset = torchvision.datasets.CIFAR10(
    root='./data', train=False, download=True, transform=transform)

test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

```

```

In [ ]: def test():
    model.eval()
    correct = 0

    with torch.no_grad():
        for data, target in test_loader:
            output = model(data)

            prediction = output.argmax(dim=1)

            correct += (prediction == target).sum().item()

    accuracy = correct / len(test_loader.dataset)
    print(f"Test Accuracy: {accuracy:.1%}")

test()

```